编译优化文章

> 23373334 彭子峻

首先展示优化前/后的测试样例运行结果。

1	2	3	4	5	6	7	8
3126	8169	8579	5402	19850350	7261242	9758	23834207
28	1861	6710	573	10605238	994053	366	152016

由于各优化间存在依赖或协同关系（如控制流简化，常数折叠可协同），故并没有测量引入某个优化后，测试样例的运行结果，上述优化后结果，是在完成所有优化后统一测量的。

对LLVM IR进行的分析

控制流分析

找出各个块的前驱/后继块。

其实现很简单，使用广度优先搜索，从入口块开始，维护访问队列，进行访问，并构建访问过程中产生的前驱/后继关系，并将结果记录在函数对象中。

循环分析

其基于“回边”进行分析。

回边的定义是，若块B存在到A的边，且块A支配块B，则形成一条回边。

我们需要找到这些基本块：

- preHeader：从循环外，进入循环的块
- header：循环头，控制循环进行
- latch：跳转到循环头的循环内部块
- exitingBlock：跳出循环的内部块
- exitBlock：跳出循环后，转移到的循环外的块

基本思路如下：

- 按照支配关系的后序遍历顺序，来检查基本块。
- 对每个基本块，检查它的所有前驱，看是否存在满足上述定义的回边。

- 一旦发现一条回边 `B -> A`， `A` 就被确定为一个循环的入口。
- 如果一个入口基本块 `A` 有多条来自不同基本块的回边，它们可能属于同一个大循环，也可能分别构成 `A` 的多个子循环。算法会处理这种情况，构建出循环的层次结构（父循环、子循环）。

支配树分析

这里参考了[Algorithms for Finding Dominators in Directed Graphs, Page 25, Algorithm 3]中的算法，对各个块的支配关系进行分析。

其基本思路是这样：

- 计算支配关系：假设某个块B被移除，再从函数入口开始遍历，找到遍历后无法到达的块，那些块就是被块B支配的块。

此外，还需要确定直接支配关系：

- 对于一个基本块 `block`，以及支配他的块的集合 `dominatorSet`：
- `block` 的直接支配者，是 `dominatorSet` 中，满足以下条件的那个块 `idom`：
  1. `idom` 支配 `block`。
  2. `idom` **不支配** `dominatorSet` 中，任何其他也支配 `block` 的块。

对LLVM IR做的机器无关优化

Mem2Reg

将用 `alloc` 指令分配的非数组变量，由内存形式，全部转换为局部变量的形式。

其依赖**支配树分析**。在各个支配边界处，插入Phi指令，并进行load/store的替换，以及变量的重命名。

死代码消除

首先定义以下指令为“有效指令”：

- 函数调用
- 函数返回
- 访存操作

若一条指令直接、或间接与有效指令产生关联，则这一指令也会被视为有效指令。

若某个块中含有有效指令，则该块为有效块；其包含的Phi指令的所有来源块也被标记为有效块。

对于有效块，其最后一条指令，即一条跳转指令，也标记为有效。

完成上述检查后，移除没有被标记为有效的指令和块。同时，需要修正基本块被移除后的控制流，即修复各块的跳转指令。

循环不变式外提

对于循环内的指令，若其计算没有直接/间接使用循环内定义的变量，则其定义变量在整个循环过程中一定不变，可将其提出循环。

对于这些被提出的指令，我们在循环的header及其前驱之间插入一个基本块，把提出的指令存在这一基本块内。

判断逻辑如下：

- 找出循环内指令定义的变量，标记之；
- 若某条指令使用了被标记的变量，则这条指令不可外提；
- 如果可外提指令没副作用（如Store, 函数调用），则可外提这条指令，到新加入的基本块中。

常数折叠

对于一个计算或比较指令，若两个操作数均为常数，则将指令定义的变量替换成执行结果对应的常数。

当然，我们需要遍历使用这一指令定义变量的指令，将他们替换为对应的常数。

乘法优化

实现了一些简单的，针对特殊乘数的情况：

- 乘数为0，直接把结果换成常量0
- 乘数为1，直接把结果换成另一乘数
- 为-1，则指令换成 `sub res, 0, anotherOperand`

若乘数为正数，则：

- 若为 2^k，则治疗换成 `res = anotherOperand << k`
- 若为 $2^x + 2^y$，则转换为 `res = (anotherOperand << x) + (anotherOperand << y)`
- 若为 $2^x - 2^y$，则转换为 `res = (anotherOperand << x) - (anotherOperand << y)`

控制流简化

对于条件为常数的跳转指令，转换为直接跳转指令，并移除控制流图中，另一跳转目标的转移边。

若某个块只有一个前驱，且这一唯一前驱只有唯一后继，则将这两个块合并。

处理只读全局变量

识别出那些只读不写的全局变量。

对于写的Store指令，直接移除。

对于Load指令，将其定义的变量替换为全局变量初始值对应的常量，并对有关的变量进行换名。

函数内联

基本思路是：

- 将基本块，按照调用指令，分为上下两块；
- 在两块之间，内联函数代码：将参数换名为调用时的实参；将调用指令定义的变量，换名为函数ret值；将return指令，换为跳转到分块下半部分的跳转指令
- 允许递归内联，但是内联历史如果出现了环，则停止递归内联。

最后结果也确实符合预期。

我预期通过重复函数内联+控制流简化+常量折叠的方法，对如testcase7中求斐波那契数列的递归调用情况进行处理，希望将调用转为常数。但是，由于控制流简化依赖控制流分析，如此重复下来，完全展开testcase7情况的的效率很低，会导致testcase2运行超时。

相信引入局部解释执行的思路，可以缓解这个问题。时间有限，我没有对这种做法进行尝试。

循环展开

采用局部执行的思路。

- 从循环的header开始解释执行
- 整数常量/在块内利用整数常量定义的指令对应的变量，视为已知值
- 对于含有不可知操作数的指令，将其保留，插入到复制的新块中
- 使用函数调用，内存Load指令定义的变量，**其值不可知。**
- 对于直接跳转指令，跳转到对应的Block，继续解释执行；条件跳转指令，进行evaluate，看条件是否可知，若不能则解释执行失败，循环不可展开。

此思路实现下来，能处理的情况十分有限。由上面过程可知，在循环嵌套的情况下，**如果内层循环不可展开，则外层均不可展开**。因此，实现该优化后，观察到对各Case的优化均不明显，甚至没有优化，悲。

可能的优化思路，是解释执行整个过程，记录内存变化状态；又或者，进一步处理需要Load/Store的情况，比如说局部/全局/静态数组，将其局部变量化。两个的实现都很棘手，我没有进一步实现。

块内公共子表达式提取（LVN）

基本思路如下：

1. 为基本块内的每条指令计算一个哈希值，该哈希值基于指令的操作类型和操作数，用于快速识别可能等价的指令。
2. 当遇到具有相同编号的新指令时，通过更精确的`numberingEquals`方法，确认两者是否真正等价。
3. 若发现等价指令，后续指令中使用的该指令的结果**被替换为先前等价的指令结果**；多余的计算指令，将通过死代码消除处理掉。
4. 检查当前指令的所有操作数，若已有对应等价替换，继续递归替换。

对目标代码做的机器无关优化

Peephole

只处理了有限的情况，如：

- add a, a, 0
- sub a, a, 0
- a = a + $zero
- 等等，详见代码实现。

对于这些，直接将他们从目标代码内移除。

目标代码生成时的乘法强度削弱

实现得很有限，处理了除数为1, -1, 2的幂次的情况。

消Phi

实现很简单，在正常消Phi的基础上，对于来源块只有一个后继的情况，直接在其末尾插入一条赋值语句。

由于LLVM IR自身是SSA形式，我们的实现**省去了先在来源处保留到临时寄存器，再在Phi指令处逐个加载**这个过程，以降低寄存器分配压力。事实证明，这么做不会引起错误，这是SSA自身性质保证的。

机器有关优化

利用CP1寄存器

CP1是MIPS架构的浮点协处理器，其拥有32个浮点寄存器，其宽度与MIPS架构物理寄存器的宽度一致。

考虑到我们的SysY语言不涉及任何浮点运算，这里选择利用这32个浮点寄存器。

使用场景按分配先后顺序排序，包括：

- SysY语言中的**全局非数组**变量，以及**非数组static**变量
- 寄存器分配过程中，溢出变量的保存位置
- 寄存器分配时，函数调用前后保存/恢复寄存器的位置

其基本思路是尽可能减少load/store，并将其换为加权仅为1的mfc1/mtc1寄存器。

寄存器分配

本编译器设计，基本复刻了《Building an Optimizing Compiler》于“寄存器分配”部分阐述的寄存器分配方法。

首先，在自LLVM IR生成MIPS代码时，我们先生成用虚拟寄存器（下称VReg）建立数据依赖的指令，再在寄存器分配过程完整后，建立VReg到架构物理寄存器（下称AReg）的映射。

具体而言，其阐述的寄存器分配方法有以下内容：

- 不划分跨块全局VReg/块内局部VReg各自可用的物理寄存器，也就是说，所有可用的AReg，有可能被分配给全局VReg，也可能分配给局部VReg；
- 先分配全局VReg的AReg，再分配局部VReg的；
- 对于全局VReg，仅对全局VReg构建冲突图，并根据之进行图着色分配；其图着色算法进行了优化，先将所有全局VReg，按照一定次序全部入栈，在弹出时再判断是否仍有颜色可用，没有才溢出。这里选择的“一定次序”，是从按秩先高后低，引用频度从高到低；
- 对于溢出的全局VReg，对于其活跃所在的基本块，在块头插入一从溢出位置load的指令，在块尾插入一save到溢出位置的指令，并将块内这一溢出的全局VReg换名为块内的一个新局部VReg，与插入的load/save关联，参与局部寄存器分配。
- 对于局部寄存器分配，将各局部VReg和全局VReg一起构建冲突图，使用这一冲突图进行图着色算法的变体：对于`秩 <= num(AReg)`的Vreg入栈，反之则不入栈；入栈的变量使用图着色算法的后续着色步骤，而没入栈的证明可能出现溢出，使用一较复杂的反向线性扫描算法分配寄存器，或进行溢出。

局部寄存器分配的内容较为复杂，单独提出来，大致说一下：

- 将在某个基本块活跃的全局Vreg,分为四类：liveStart（从块头活跃，到块内某处不再活跃）、liveEnd（在块内某处定义，一直活跃到块尾）、liveTransparent（没在块内定义/使用，但始终在块中活跃的VReg），liveThrough（在块内各处都被定义/使用），可以通过反向线性扫描，基于活跃变量分析，得出结果。
- 对于某个局部VReg,若其定义晚于一liveStart全局变量，则可以复用其分配到的AReg；
- liveStart的AReg，可在后续反向线性扫描分配的过程中完成分配。
- 后续的分配过程，其顺序为：先复用liveStart，再反向线性扫描分配。

- 反向线性扫描过程中，若遇到定义点，则释放对应AReg；需要注意，**liveStart对应的AReg不允许回收**，遇到初次使用/定义点，则进行寄存器分配。
- 若无寄存器可用，则选取一已经分配好的VReg,进行溢出：对于这一victim VReg，在其正向最后一次被定义/使用处后，插入Store到溢出位置的指令；在当前反向扫描到的指令之后，插入从溢出位置load的指令。**需要注意的是，若当前待分配的Vreg定义点，早于victim VReg的最后定义/使用点，则不应溢出这个victim VReg.**

这个寄存器分配方法确实有效，我通过关闭溢出允许使用CP1寄存器选项的方法，确认此方法造成的溢出很少。

不过实现它的代价是什么呢?

- 时间：做这玩意，包括修正原文伪代码错误/debug时间，花了我一周有余；
- 函数调用前后，保存/恢复现场的开销：传统方法里面，区分全局/局部变量可用寄存器的实现，可以只保存全局变量对应的寄存器；而在我们现在的实现中，我们无法确认，哪些函数认为哪些是全局/局部变量，故传统的Caller Save方法**只能保存所有分配了的AReg，其带来的load/save开销极大。** 这就是为什么我们要利用CP1寄存器缓解这个问题。改成调用处保存太过复杂，我们没有尝试。

那么，这些优化在现实真的可行吗?

可以看到，为了让MARS模拟出的周期尽可能少，我们做了一些现实情况不该做的事，包括但不限于：

- 无节制的函数内联，循环展开；这在现实会导致指令取指次数激增，且会极大影响指令Cache的缓存效果；但是，MARS既没有模拟指令取指的开销，也没有模拟Cache，函数内联/循环展开的代价被隐藏了。
- 破坏调用栈：比如把部分现场保留在CP1寄存器内；这对于调试器是灾难。
- 把某些一般约定应保存在内存中的变量（如，全局变量，静态变量），放到寄存器里。我们此时有一个隐式假设，是内存**仅本机可用，不会被其他设备改变**。现实情况下，这个假设并不成立，这么做是很危险的。

上述是我对我做的优化的一点拓展思考。（笑）